Module 12, Milestone #6: Build Data Pipeline

Group 08:
Raquel Amarins, Student 2, Student 3

Objective:
The objective of this exercise was to build an end-to-end AWS data pipeline that ingests, transforms, and analyzes 311 calls and crash datasets to uncover meaningful relationships between service requests and roadway incidents.

Tools used:

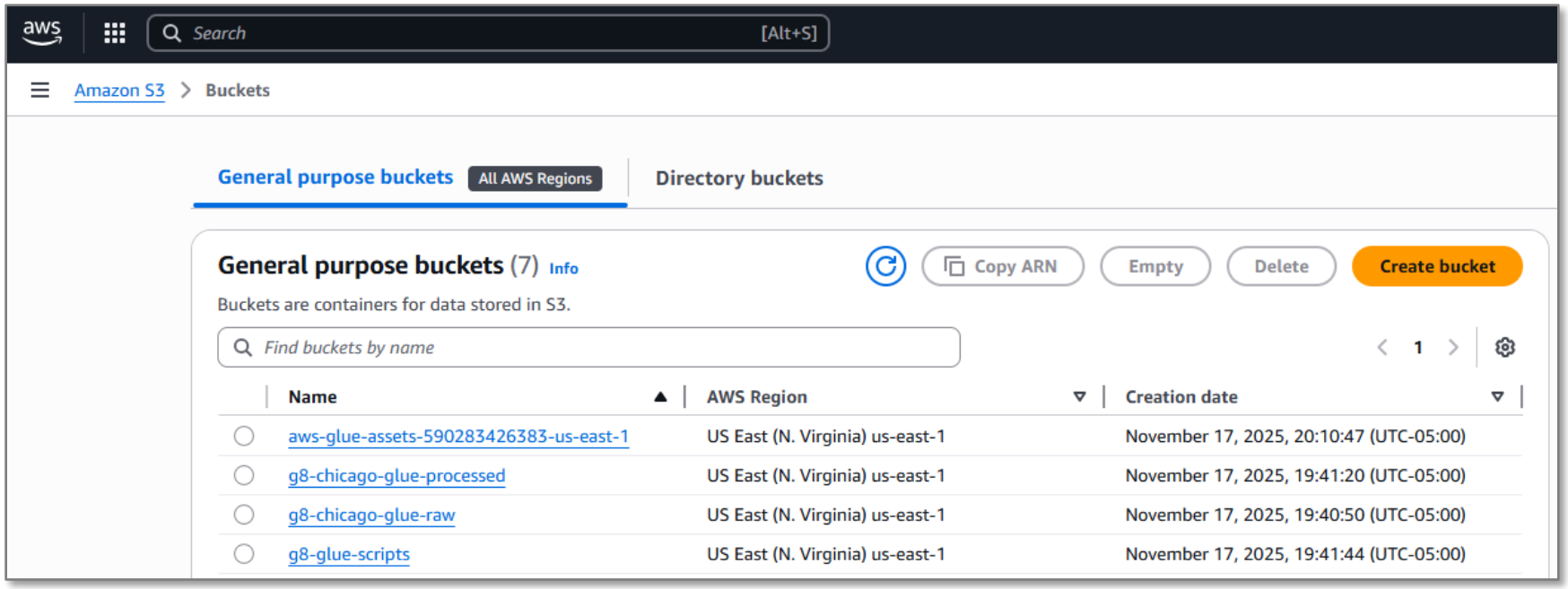
Tools	Implementation	Purpose
AWS S3	Store extracted and transformed data	Store initial raw data, store data extracted from the raw data through transformation, serve as input for data exploration in Athena
Glue Studio	Used interactive Notebook ETL Job to run Python queries	ETL Jobs enabled data extraction and data manipulation, as well as sending the data to S3 in json and parquet format
Athena	Data exploration	Tested queries after creating them, and analyzed patterns and vulnerabilities before proceeding to the next stage

Programming Languages:
Python (all Notebook Jobs) and PrestoSQL/Trino for all Athena queries.

Process:

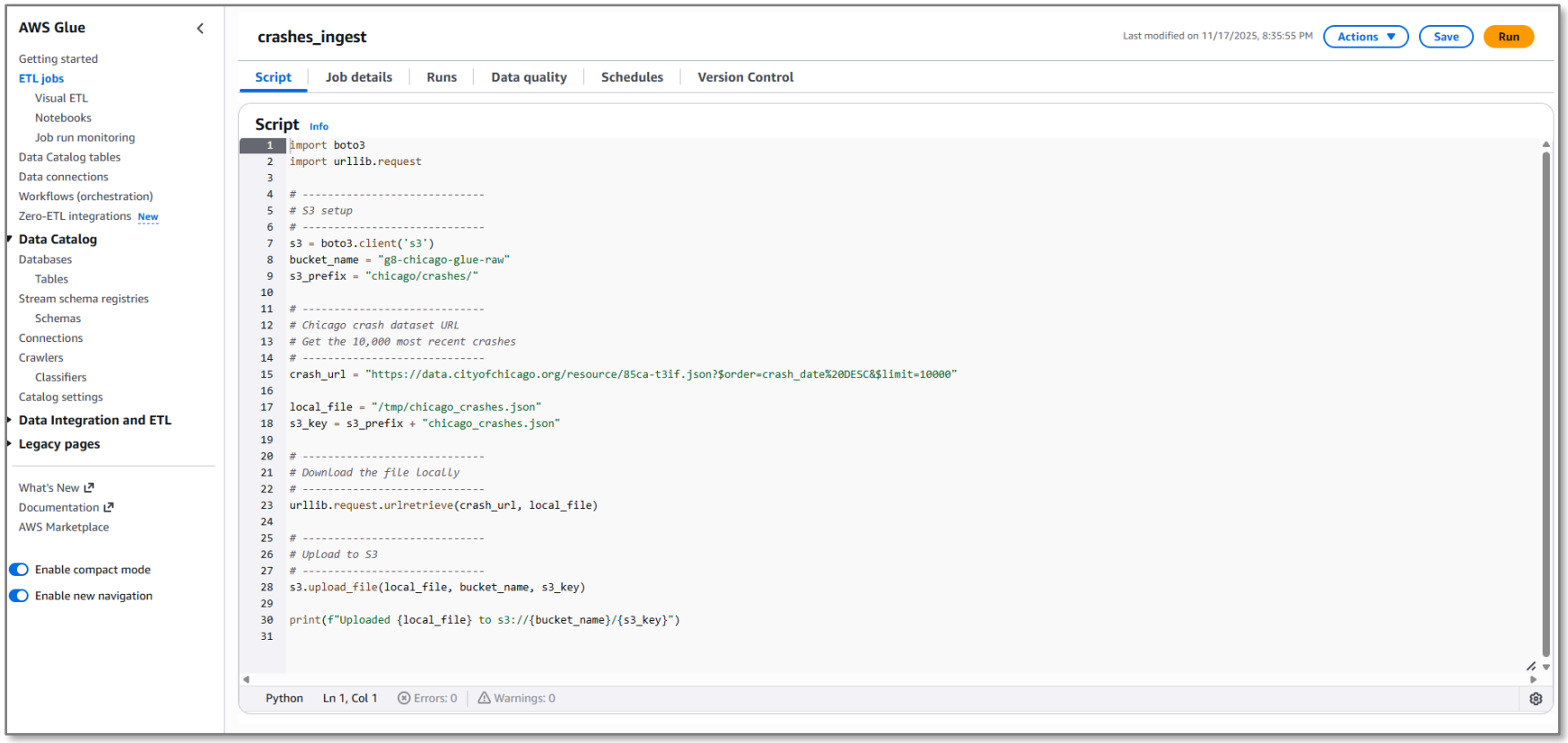
1) Ingest data:
   - Tools used: AWS Glue and S3

   Pre-work: create buckets in S3



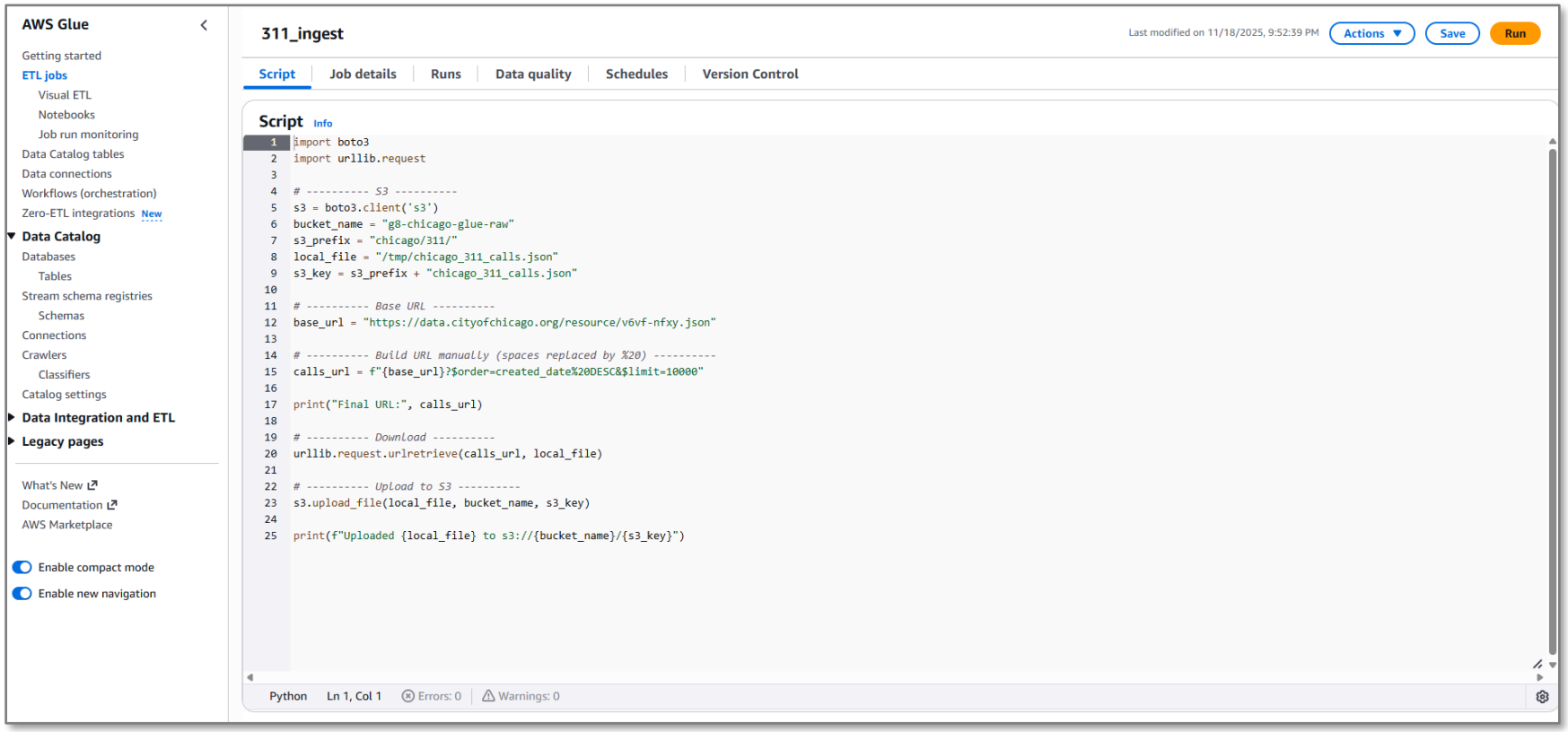
1a) Ingest crashes data.

Tools used: AWS Glue (extract) and S3 (storage).



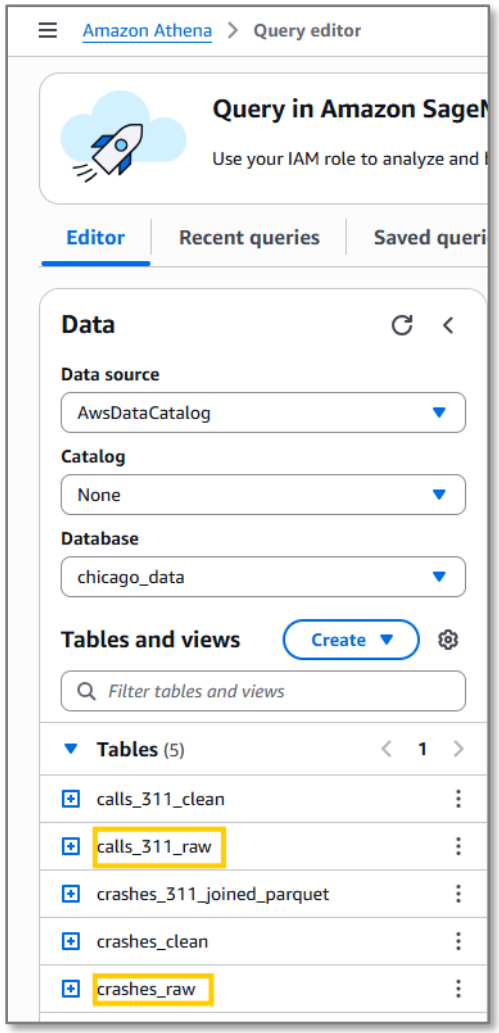
1b) Ingest 311_calls data.

Tools used: AWS Glue (extract) and S3 (storage).



Loaded raw tables for exploration in Athena:

Tool used: Athena.



2) Transform

- Tools used: Glue Notebooks

Your jobs (10) Info

Filter jobs by property

	Job name		Type	Created by
	join_address_v1		Glue ETL	Notebook
	crashes_exploration		Glue ETL	Notebook
	calls_exploration		Glue ETL	Notebook
	311_calls_clean	3	Glue ETL	Notebook
	crashes_clean_v2	4	Glue ETL	Notebook
	filtered_sr_types	2	Python shell	Script
	service_request_type	1	Python shell	Script
	311_ingest		Python shell	Script
	checking_all_data		Python shell	Script
	crashes_ingest		Python shell	Script

2a) [1] Identified all unique service request types:

service_request_type

Script | Job details | Runs | Data quality | Schedules | Version Control

Script Info

```python
import boto3
import urllib.request
import json

# ---------- S3 setup ----------
s3 = boto3.client('s3')
bucket_name = "g8-chicago-glue-raw"
s3_prefix = "chicago/311/"
local_file = "/tmp/311_sr_type_unique.json"
s3_key = s3_prefix + "311_sr_type_unique.json"

# ---------- API ----------
base_url = "https://data.cityofchicago.org/resource/v6vf-nfxy.json"
calls_url = f"{base_url}?$order=created_date%20DESC&$limit=50000"

# ---------- Download ----------
response = urllib.request.urlopen(calls_url)
data = json.loads(response.read())

# ---------- Extract unique sr_type ----------
unique_values = sorted(set(item['sr_type'] for item in data if 'sr_type' in item))

# ---------- Save to local file ----------
with open(local_file, 'w') as f:
    json.dump(unique_values, f, indent=2)

# ---------- Upload to S3 ----------
s3.upload_file(local_file, bucket_name, s3_key)

print(f"Unique sr_type values saved to s3://{bucket_name}/{s3_key}")
```

2b) [2] Filtered just for service requests pertinent to the project:

filtered_sr_types

Script | Job details | Runs | Data quality | Schedules | Version Control

Script Info

```python
import boto3
import json
import tempfile

s3 = boto3.client('s3')

bucket_raw = "g8-chicago-glue-raw"
bucket_processed = "g8-chicago-glue-processed"

raw_key = "chicago/311/chicago_311_calls.json"
output_prefix = "chicago/311/filtered/"
output_file = "filtered_311_calls.json"
output_key = output_prefix + output_file

allowed_types = [
    "Alley Light Out Complaint",
    "Alley Pothole Complaint",
    "Ice and Snow Removal Request",
    "Pothole in Street Complaint",
    "Protected Bike Lane - Debris Removal",
    "Sign Repair Request - Do Not Enter Sign",
    "Sign Repair Request - One Way Sign",
    "Sign Repair Request - Stop Sign",
    "Snow - Object/Dibs Removal Request",
    "Street Cleaning Request",
    "Street Light Out Complaint",
    "Street Light Pole Damage Complaint",
    "Street Light Pole Door Missing Complaint",
    "Traffic Signal Out Complaint",
    "Tree Debris Clean-Up Request",
    "Viaduct Light Out Complaint",
    "Water On Street Complaint"
]
```

Python Ln 1, Col 1 Errors: 0 Warnings: 0

2c) [3] Created **clean calls** table with pertinent rows and columns for the project, while transforming data types (date) and concatenating columns (address)

311_calls_clean

Last modified on 11/21/2025, 3:07:47 PM

Notebook | Script | Job details | Runs | Data quality | Schedules | Version Control

Markdown

```
|SR25-02128352|11          |Street Light Out Complaint|60620   |S HALSTED ST  |2025-11-18 19:49:57|2025-11-18       |
|SR25-02128341|11          |Alley Light Out Complaint |60620   |S EMERALD AVE |2025-11-18 19:47:25|2025-11-18       |
+-------------+------------+--------------------------+--------+--------------+-------------------+-----------------+
only showing top 5 rows
```

[12]:
```
df_311_clean = df_311_clean.select(
    "sr_number",
    "created_date_only",
    "created_month",
    "sr_type",
    "full_address",
    "zip_code"
)
```

[13]:
```
df_311_clean.show(5, truncate=False)
```

```
+-------------+-----------------+-------------+-------------------------+--------------+--------+
|sr_number    |created_date_only|created_month|sr_type                  |full_address  |zip_code|
+-------------+-----------------+-------------+-------------------------+--------------+--------+
|SR25-02128377|2025-11-18       |11           |Street Light Out Complaint|W 112TH PL    |60628   |
|SR25-02128374|2025-11-18       |11           |Street Light Out Complaint|E 56TH ST     |NULL    |
|SR25-02128362|2025-11-18       |11           |Street Light Out Complaint|W JACKSON BLVD|NULL    |
|SR25-02128352|2025-11-18       |11           |Street Light Out Complaint|S HALSTED ST  |60620   |
|SR25-02128341|2025-11-18       |11           |Alley Light Out Complaint |S EMERALD AVE |60620   |
+-------------+-----------------+-------------+-------------------------+--------------+--------+
only showing top 5 rows
```

[14]:
```
df_311_clean.printSchema()
df_311_clean.count()
```

```
root
 |-- sr_number: string (nullable = true)
 |-- created_date_only: date (nullable = true)
 |-- created_month: string (nullable = true)
 |-- sr_type: string (nullable = true)
 |-- full_address: string (nullable = false)
 |-- zip_code: string (nullable = true)

733
```

2d) [4] Created **crashes table** with pertinent columns to the project while transforming data types (date) and concatenating columns (address)

crashes_clean_v2

Last modified on 11/21/2025, 2:59:08 PM | Stop notebook | Download Notebook | Actions | Save | Run

Notebook | Script | Job details | Runs | Data quality | Schedules | Version Control

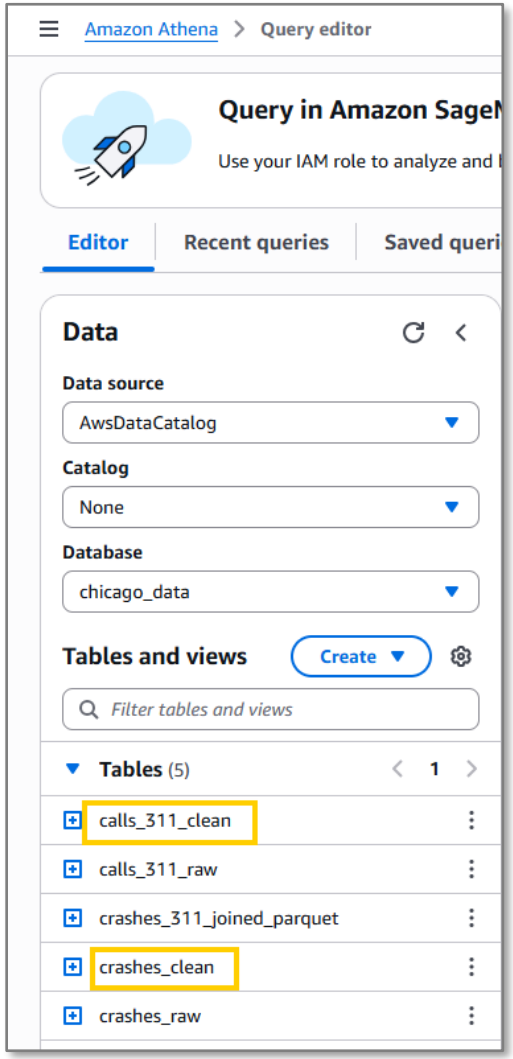
Markdown | Glue PySpark

[11]:
```
df_crashes_clean = df_crashes_clean.drop("street_direction", "street_name")
```

[12]:
```
df_crashes_clean.show(5, truncate=False)
df_crashes_clean.printSchema()
```

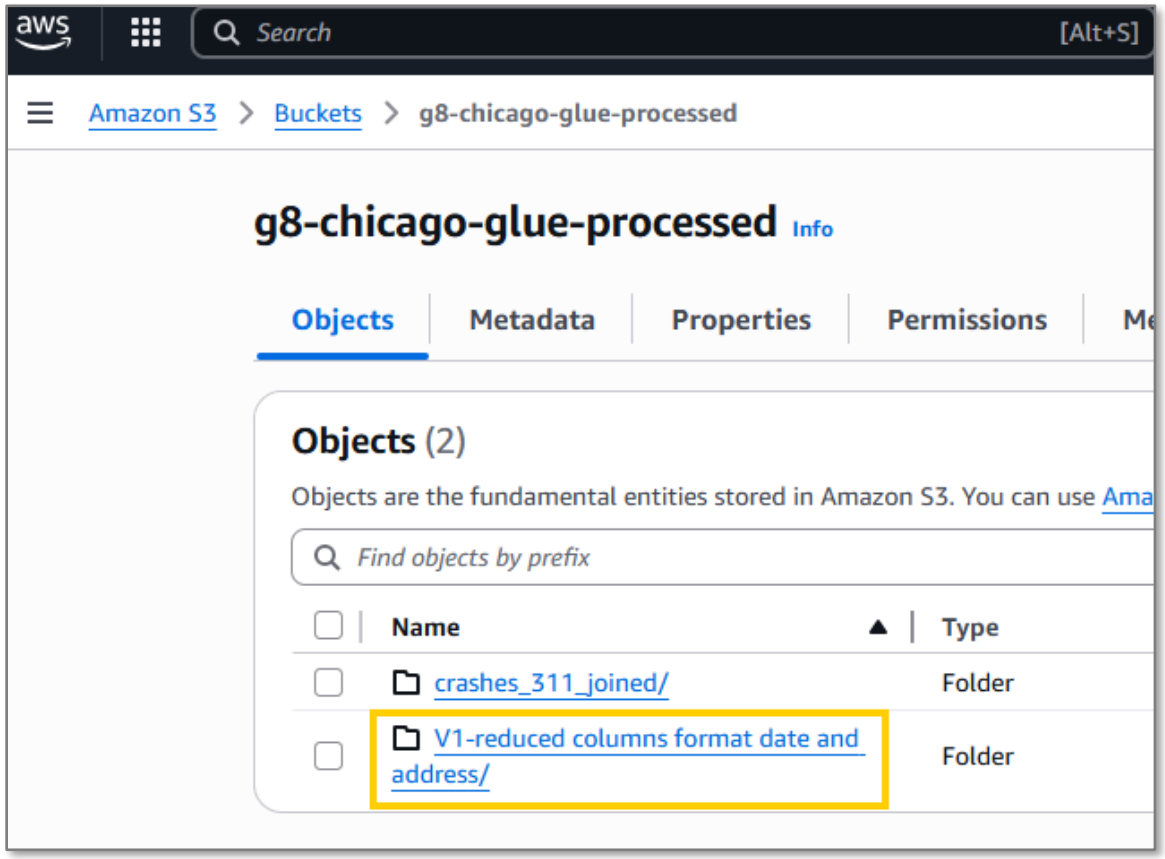
```
+-----------------------
```

3) Loaded clean data into Athena for exploration:

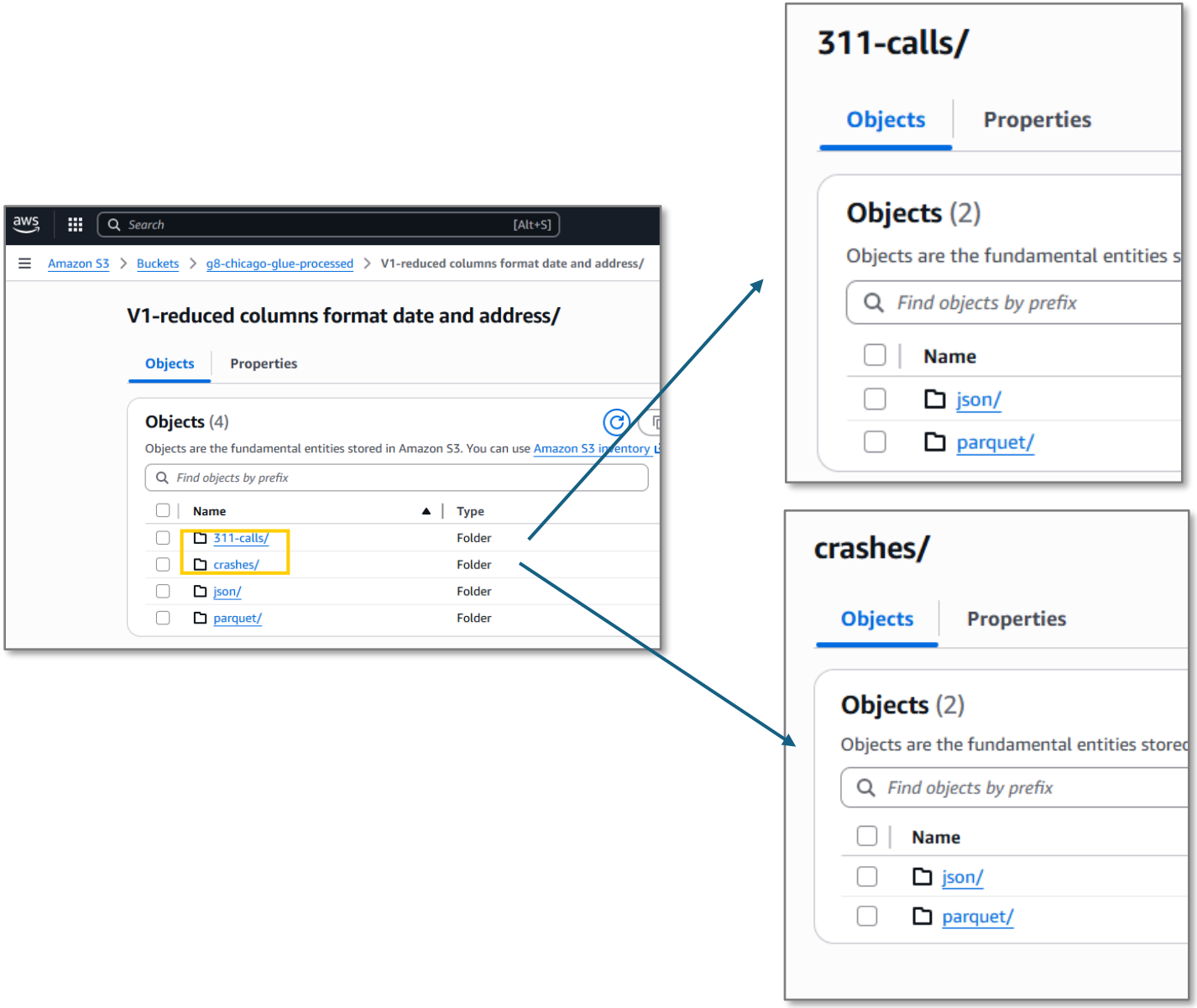


4) Saved final transformed files in S3:

Tool used: AWS S3



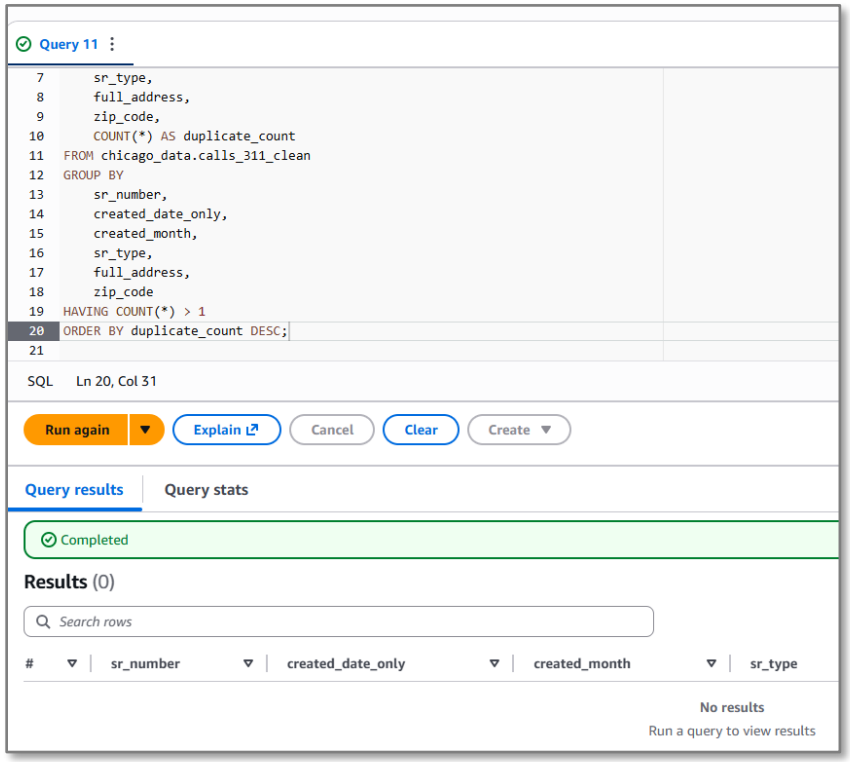
4a) At each transformation stage, we generated both a Parquet file for efficient querying and analysis, and a JSON file as a clean, structured backup to support further transformations if needed.



5) Using Athena to explore data before final join:

5a) 311 calls table:

- There are no duplicates



- Identified nulls in the zip code column (future proofing: keep vulnerability in mind if there are intentions of using zip code in project's dashboard)
- There are no nulls in the date or address columns

```sql
SELECT
    COUNT(*) AS total_rows,
    COUNT_IF(sr_number IS NULL)          AS null_sr_number,
    COUNT_IF(created_date_only IS NULL)  AS null_created_date_only,
    COUNT_IF(created_month IS NULL)      AS null_created_month,
    COUNT_IF(sr_type IS NULL)            AS null_sr_type,
    COUNT_IF(full_address IS NULL)       AS null_full_address,
    COUNT_IF(zip_code IS NULL)           AS null_zip_code
FROM chicago_data.calls_311_clean;
```

SQL Ln 22, Col 1

Run again | Explain | Cancel | Clear | Create

Reuse query results
up to 60 minutes ago

Query results | Query stats

Completed — Time in queue: 100 ms — Run time: 452 ms — Data scanned: 10.44 KB

Results (1)

Copy | Download results CSV

Search rows

1

#	total_rows	null_sr_number	null_created_date_only	null_created_month	null_sr_type	null_full_address	null_zip_code
1	733	0	0	0	0	0	287

➢ No leading/trailing spaces in the address column:

Query 11 | Query 12

```sql
SELECT
    COUNT(*) AS total_rows,
    COUNT_IF(sr_number IS NULL)          AS null_sr_number,
    COUNT_IF(created_date_only IS NULL)  AS null_created_date_only,
    COUNT_IF(created_month IS NULL)      AS null_created_month,
    COUNT_IF(sr_type IS NULL)            AS null_sr_type,
    COUNT_IF(full_address IS NULL)       AS null_full_address,
    COUNT_IF(zip_code IS NULL)           AS null_zip_code
FROM chicago_data.calls_311_clean;

SELECT *
FROM chicago_data.calls_311_clean
WHERE full_address != TRIM(full_address);
```

SQL Ln 32, Col 1

Run again | Explain | Cancel | Clear | Create

Query results | Query stats

Completed Tim

Results (0)

Search rows

#	sr_number	created_date_only	created_month	sr_type

No results

Run a query to view results

5b) crashes table:

- There are no duplicate rows

Query 11 | Query 12

```sql
    crash_record_id,
    crash_date,
    crash_month,
    weather_condition,
    lighting_condition,
    roadway_surface_cond,
    road_defect,
    crash_type,
    prim_contributory_cause,
    full_address,
    COUNT(*) AS duplicate_count
FROM chicago_data.crashes_clean
GROUP BY
    crash_record_id,
    crash_date,
```

SQL Ln 3, Col 1

Run again | Explain | Cancel | Clear | Create

Query results | Query stats

Completed Time in queue: 112 ms Run time: 722 ms

Results (0) Copy

Search rows

#	crash_record_id	crash_date	crash_month	weather_condition	lighting_condition	roadway_surface_cond	road_defect	crash_type

No results

Run a query to view results

➢ There are no nulls in any of the columns

Query 11 | Query 12

```sql
SELECT
    COUNT(*) AS total_rows,
    COUNT_IF(crash_record_id IS NULL)          AS null_crash_record_id,
    COUNT_IF(crash_date IS NULL)               AS null_crash_date,
    COUNT_IF(crash_month IS NULL)              AS null_crash_month,
    COUNT_IF(weather_condition IS NULL)        AS null_weather_condition,
    COUNT_IF(lighting_condition IS NULL)       AS null_lighting_condition,
    COUNT_IF(roadway_surface_cond IS NULL)     AS null_roadway_surface_cond,
    COUNT_IF(road_defect IS NULL)              AS null_road_defect,
    COUNT_IF(crash_type IS NULL)               AS null_crash_type,
    COUNT_IF(prim_contributory_cause IS NULL)  AS null_primary_cause,
    COUNT_IF(full_address IS NULL)             AS null_full_address
FROM chicago_data.crashes_clean;
```

SQL Ln 31, Col 1

Run again | Explain | Cancel | Clear | Create

Reuse query results
up to 60 minutes ago

Query results | Query stats

Completed — Time in queue: 106 ms — Run time: 551 ms — Data scanned: 1.28 MB

Results (1)

Copy | Download results CSV

Search rows

1

#	total_rows	null_crash_record_id	null_crash_date	null_crash_month	null_weather_condition	null_lighting_condition	null_roadway_surface_cond	null_road_c
1	10000	0	0	0	0	0	0	0

➢ No leading/trailing spaces in the address column:

Query 11 | Query 12

```
    COUNT_IF(crash_date IS NULL)              AS null_crash_date,
    COUNT_IF(crash_month IS NULL)             AS null_crash_month,
    COUNT_IF(weather_condition IS NULL)       AS null_weather_condition,
    COUNT_IF(lighting_condition IS NULL)      AS null_lighting_condition,
    COUNT_IF(roadway_surface_cond IS NULL)    AS null_roadway_surface_cond,
    COUNT_IF(road_defect IS NULL)             AS null_road_defect,
    COUNT_IF(crash_type IS NULL)              AS null_crash_type,
    COUNT_IF(prim_contributory_cause IS NULL) AS null_primary_cause,
    COUNT_IF(full_address IS NULL)            AS null_full_address
FROM chicago_data.crashes_clean;

SELECT *
FROM chicago_data.crashes_clean
WHERE full_address != TRIM(full_address);
```

SQL Ln 45, Col 1

Run again | Explain | Cancel | Clear | Create

Query results | Query stats

Completed

Results (0)

Search rows

#	crash_record_id	crash_date	crash_month	weather_condition	lighting_condition	roadway

No results

Run a query to view results

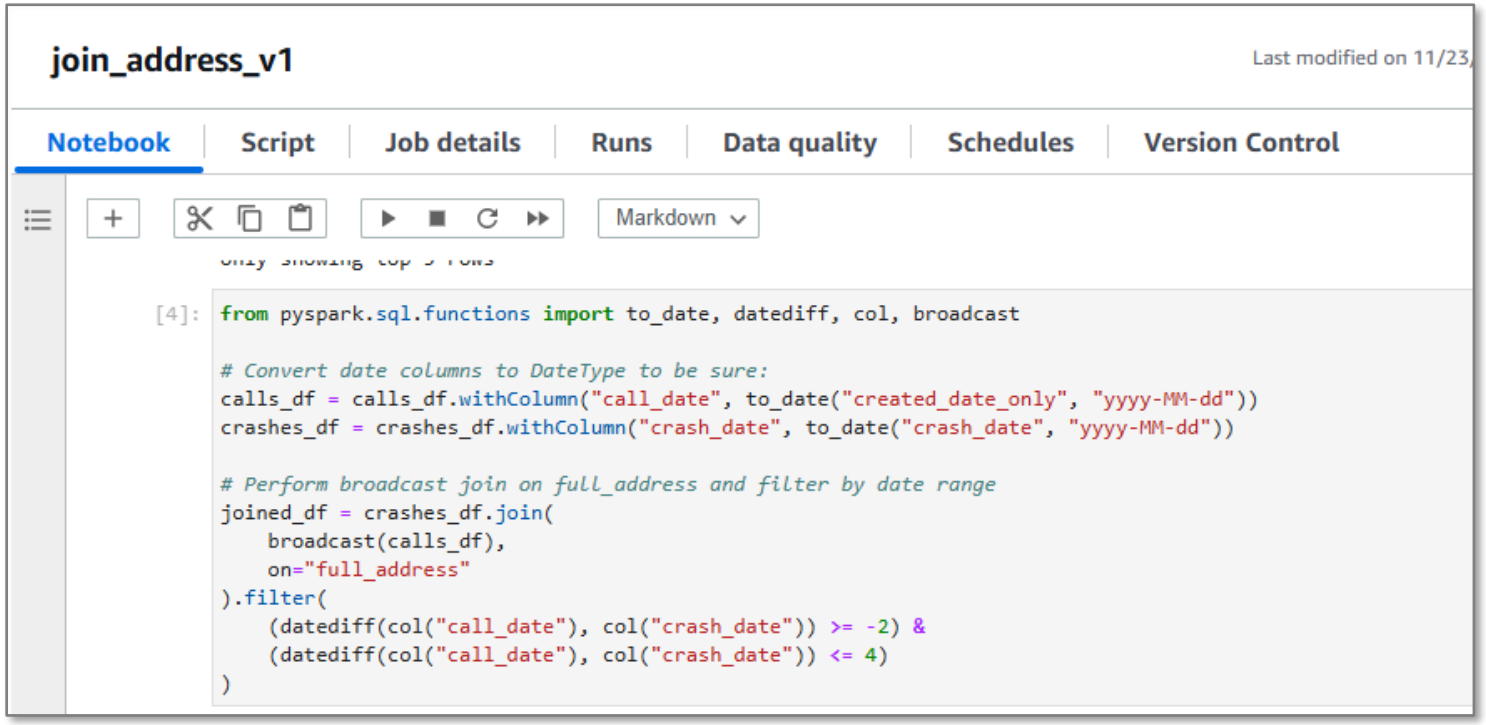
6) Joining tables to fulfill statement problem

Tools used: AWS Glue

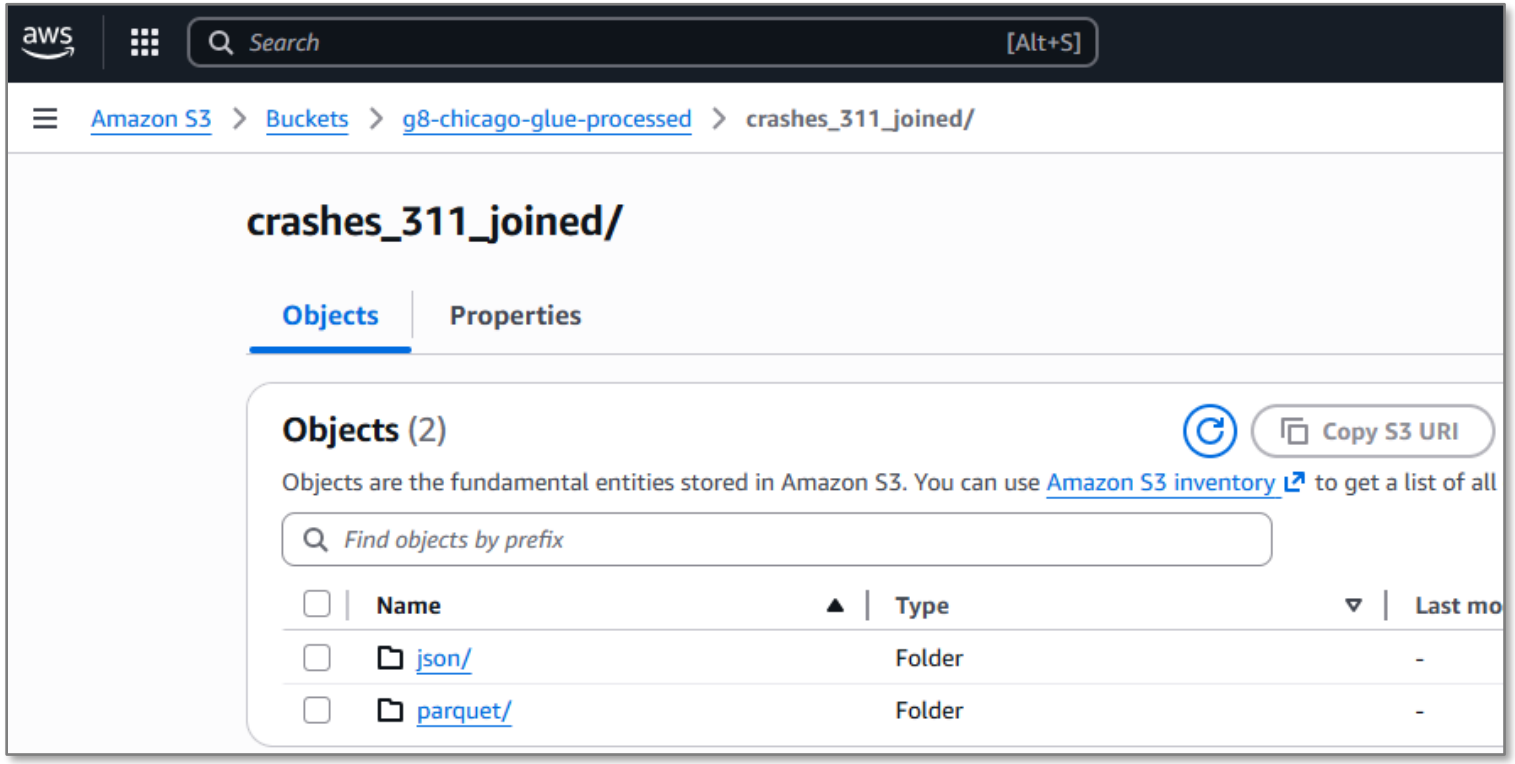
This join links car crash records with 311 service requests based on:

- Exact address match (full_address)
- Temporal proximity: crashes that occurred within a window of 2 days before to 4 days after the service request date

We use a **broadcast join** to efficiently match the small 311 dataset (~700 rows) with the larger crash dataset. This approach ensures high spatial and temporal precision while maintaining performance.



7) Saving the join table in S3:



8) Visualizing join table in Athena using the Parquet file created:

➢ Preview table:

```
SELECT * FROM "chicago_data"."crashes_311_joined_parquet" limit 10;
```

SQL Ln 1, Col 1

Run again | Explain | Cancel | Clear | Create

Reuse query results up to 60 minutes ago

Query results | Query stats

Completed — Time in queue: 214 ms — Run time: 740 ms — Data scanned: 94.43 KB

Results (10)

Copy | Download results CSV

Search rows

1

#	crash_record_id	sr_number	sr_type
1	2dd8fb1de8a130050880e91d9e039f3a338dbd8f8e8a84432a8c8e03b61d8fb1fcae9143d5adf938527359843dc14769094baea1f8f6c99ac9c8dc0746c50571	SR25-02115907	Street Cleaning Request
2	2dd8fb1de8a130050880e91d9e039f3a338dbd8f8e8a84432a8c8e03b61d8fb1fcae9143d5adf938527359843dc14769094baea1f8f6c99ac9c8dc0746c50571	SR25-02125033	Sign Repair Request - Stop Sign
3	2dd8fb1de8a130050880e91d9e039f3a338dbd8f8e8a84432a8c8e03b61d8fb1fcae9143d5adf938527359843dc14769094baea1f8f6c99ac9c8dc0746c50571	SR25-02128060	Street Light Out Complaint
4	afe079dd863554293afc719fdadf66bc27ba450c5ec9f0c266e32da48402baf6e828da248b2b8904a046eff7039346c8e2cbe241e7dc9e0c8278b5d53a86c4aa	SR25-02120448	Traffic Signal Out Complaint
5	afe079dd863554293afc719fdadf66bc27ba450c5ec9f0c266e32da48402baf6e828da248b2b8904a046eff7039346c8e2cbe241e7dc9e0c8278b5d53a86c4aa	SR25-02120667	Street Light Out Complaint
6	8b47d1984b38405dd96fab4b8b55ffed464c366c0e93396491bbf17648803a5e5a37cd442e0af5fcea5b44786a2cbb93d0096dd9e2b2d12cfdb66c7ca0e22d90	SR25-02121821	Traffic Signal Out Complaint

➢ Working with selected columns

```sql
SELECT * FROM "chicago_data"."crashes_311_joined_parquet" limit 10;

SELECT
    sr_number,
    sr_type,
    call_date,
    crash_date,
    full_address,
    lighting_condition,
    prim_contributory_cause,
    road_defect,
    roadway_surface_cond,
    weather_condition
FROM chicago_data.crashes_311_joined_parquet;
```

SQL Ln 3, Col 1

Run again | Explain | Cancel | Clear | Create

Reuse query up to 60 minutes

Query results | Query stats

Completed — Time in queue: 97 ms — Run time: 553 ms — Data scanned: 12.4

Results (1,739)

Copy | Download results

Search rows

1 ...

#	sr_number	sr_type	call_date	crash_date	full_address	lighting_condition	prim_contributory_cause
1	SR25-02115907	Street Cleaning Request	2025-11-17	2025-11-17	N SACRAMENTO AVE	DARKNESS, LIGHTED ROAD	EQUIPMENT - VEHICLE CONDITION
2	SR25-02125033	Sign Repair Request - Stop Sign	2025-11-18	2025-11-17	N SACRAMENTO AVE	DARKNESS, LIGHTED ROAD	EQUIPMENT - VEHICLE CONDITION
3	SR25-02128060	Street Light Out Complaint	2025-11-18	2025-11-17	N SACRAMENTO AVE	DARKNESS, LIGHTED ROAD	EQUIPMENT - VEHICLE CONDITION
4	SR25-02123055	Traffic Signal Out Complaint	2025-11-18	2025-11-17	W PERSHING RD	DARKNESS, LIGHTED ROAD	UNABLE TO DETERMINE
5	SR25-02126107	Pothole in Street Complaint	2025-11-18	2025-11-17	W PERSHING RD	DARKNESS, LIGHTED ROAD	UNABLE TO DETERMINE
6	SR25-02118518	Traffic Signal Out Complaint	2025-11-17	2025-11-17	N LAVERGNE AVE	DARKNESS, LIGHTED ROAD	FOLLOWING TOO CLOSELY

Conclusion:

This analysis identified 1,739 instances in which a 311-service request was reported on a street where one or more traffic crashes occurred either shortly before or shortly after the call. This overlap highlights a meaningful relationship between public service complaints and roadway safety conditions.

By leveraging AWS tools—including S3, Glue Studio, and Athena—we were able to efficiently ingest, clean, transform, and organize large volumes of data from multiple sources. These steps enabled the development of a data-driven dashboard that demonstrates the value of the 311 service in identifying potential safety concerns and supporting proactive city operations.